WashKrew

Claude + Aman: iPhone test guide

28 October 2026 | Customer and Captain | GitHub staging

This is the current start-to-finish guide: pull the source, build both apps, create the accounts, approve the Captain, book and pay, perform the wash, track it, approve/rate it, and check cancellation/refund and device recovery.

Application baseline: `c1d1ad0`, plus the commit containing this guide and its PDF. Always record `git rev-parse HEAD` after pulling; do not check out the older baseline alone. Repo: https://github.com/desicode-blip/wheelio-Both-Apps

Read these facts before any test

1. Normal apps use Firebase project `wheelio-apps`. **The hosted gateway is LIVE** on the separate WashKrew merchant. Payment-mode labels in older reports or screenshots are historical. A successful live checkout charges real money.
2. Training videos, quiz and refresher entry points are **hidden and not required**. There are three KYC stages: Aadhaar, selfie and payout details. An authorized human admin must approve the application before the Captain can work.
3. Direct credit-card checkout is excluded. UPI, debit cards and netbanking are allowed. The owner allows a credit card linked inside a UPI app.
4. Keep the staging designs, bundle IDs and Firebase project. Do not create QA approvals, fixed OTPs, fake training, fake GPS or fake payments in production.
5. Windows checks passed: 121 backend/rules tests, 6 Captain tests and the latest Captain release build/analysis. These do not prove Xcode, signing or iPhone use.

Read in this order	Purpose
Pages 2-3	Claude instructions, Mac build, signing and installation
Pages 4-6	Customer account, Captain account/KYC, admin approval
Pages 7-10	Booking, live payment, assignment, tracking, wash approval
Pages 11-12	Cancellation/refund, disputes, permissions and settings
Pages 13-14	Failure diagnosis, evidence and final results

Passwords, SMS codes, real payment authorization and sensitive human decisions belong to Aman/the responsible person. Never put secrets or private KYC in this PDF, Git, a shared report or a chat.

2. Claude: start here on the Mac

Guide Aman through one numbered section at a time. State what he should tap, what should happen and what evidence to record. Wait for his observed result before marking a device case PASS. A missing prerequisite is BLOCKED, not PASS.

Working instructions for Claude Code

1. Read this complete guide, root `AGENTS.md`, `CLAUDE.md`, `docs/COMPLETION-2026-10-28.md` and `docs/MAC-HANDOFF.md`. The current hidden- training/admin-approval and LIVE-payment decisions supersede old test-only and training-placeholder instructions.
2. Inspect the checkout before editing. Preserve local Mac work. Fix functionality inside the existing Flutter screens; do not redesign, replace UI, change palette/assets, restore a payment banner or generate Android.
3. First reproduce each failure. Record expected/actual behavior, the screen, source commit and the relevant redacted error. Fix the smallest cause and rerun that case. Commit actual source/lock fixes. Push only when Aman asks.
4. Let Aman enter passwords/OTPs, approve real charges and perform admin identity/ payout eligibility decisions. Do not make those decisions or simulate success.

Get the latest staging source

For a new checkout:

```
git clone https://github.com/desicode-blip/wheelio-Both-Apps.git
cd wheelio-Both-Apps
git switch staging
git pull --ff-only origin staging
git rev-parse HEAD
```

For an existing checkout, run `git status --short` first, preserve local changes, then `git fetch origin staging`, `git switch staging` and `git pull --ff-only origin staging`. If fast-forward fails, inspect divergence; do not use a hard reset or force-push to erase work.

The active native apps are `apps/flutter/customer` and `apps/flutter/captain`. The SwiftUI `apps/ios/Wheelio.xcodeproj`, old Next.js apps and `legacy/` are references, not this acceptance build. Normal builds need no Razorpay secrets, Firebase owner CLI login, credential PDFs or Apple private key in app source.

3. Prepare, sign and install both apps

Mac and phone preparation

1. Use the tested Flutter 3.38.6 SDK with its locked Dart dependencies, Xcode with iOS platform support and CocoaPods. Run `flutter doctor -v`; address actual Xcode/iOS findings. Aman completes Apple login, license and account checks.
2. Connect and unlock the iPhone, trust the Mac and enable Developer Mode if requested. iOS deployment target is 15+. Xcode must support that phone's iOS.
3. From the repository root, prepare both Flutter workspaces:

```
bash apps/flutter/tool/run-ios.sh --prepare
open apps/flutter/customer/ios/Runner.xcworkspace
open apps/flutter/captain/ios/Runner.xcworkspace
```

In each workspace choose Runner, Signing & Capabilities and the owning Apple team. Saved team ID: `8CCQQM7TP3`; verify access rather than changing bundle IDs.

App	Preserve this bundle ID
Customer	`com.amanmaheshwari.wheelio.customer`
Captain	`com.amanmaheshwari.wheelio.captain`

Enable/verify Push Notifications capability and Remote notifications background mode for both. The owning team's APNs key must be configured in the existing Firebase project's Cloud Messaging settings. Private signing material stays out of Git. Missing team/key access is an Apple setup blocker, not a reason to fake push.

```
bash apps/flutter/tool/run-ios.sh
# With several connected iPhones, select one explicitly:
bash apps/flutter/tool/run-ios.sh --device IPHONE_ID
```

The script builds/installs both real Release apps. Optional `--no-codesign` compiles without signing/installing. Do not use `--demo` for acceptance. Open each app after disconnecting Xcode; confirm both cold-launch normally.

If a pod fails, retain its exact error and update only the named incompatibility. Do not delete all locks, broadly upgrade dependencies or recreate the iOS project. Commit successfully resolved Mac pod/project changes and record both app versions.

4. Test roles and Customer account

Set up the people and devices

Use **two different real consenting mobile numbers**: one Customer and one Captain. A Captain cannot accept their own Customer booking; the same Firebase user ID is excluded from availability/assignment. Reusing one number in both roles is not a valid end-to-end booking test.

Both apps can be installed on Aman's one iPhone. Use the numbers' actual holders to receive SMS. For simultaneous tracking, keep Captain on the iPhone and open Customer on the Mac at https://wheelio-apps.web.app/app/ using the Customer number. Also test the native Customer app separately on the iPhone. Two phones are preferable for simultaneous native push/location checks, but not needed to begin. Do not call one-phone foreground switching a continuous-tracking pass.

Customer: create the account from scratch

1. Open **WashKrew** Customer. Use Next or Skip on welcome slides. The real app must not show a DEMO/sample-data label or sample bookings.
2. Open **Profile -> Log in**. Enter the Customer mobile number and tap the SMS code action. Aman enters the actual OTP only in the app. No fixed demo code.
3. Check edit-number/back behavior and one incorrect-code error. Respect resend cooldown; request another SMS only when the app allows it. An expired/wrong code must not create an authenticated account or show success.
4. Enter the requested name and Continue. If the account already exists, confirm its real profile returns instead of inventing a new one. Reopen the app and confirm login/profile persistence.
5. Visit Home, Bookings, Vehicles and Profile. Empty data must be empty rather than sample jobs. Check profile photo capture/cancel/replace if offered.
6. Check Help/support and Terms/privacy. Missing approved contacts/policies are known business inputs: record BLOCKED if still absent, not a working-link pass.

Expected: real SMS sign-in and profile persistence; no phone/OTP leaks, duplicate accounts, pretend bookings or dependency on the Captain's login session.

5. Captain account, KYC and work hours

Open **WashKrew Captain**. Choose English, Hindi or Telugu, finish/skip welcome slides and use Sign up or Go online. Guests may browse, but starting work must lead to signup/KYC, never genuine Online or real job acceptance.

1. Enter the Captain's name and different mobile number. Get the SMS and enter the actual code in this app. Confirm the account opens **offline/unapproved**.
2. Open Quick KYC. It must say **3 steps to take jobs**: Aadhaar, Your photo, Bank or UPI. No training row, video, quiz or refresher entry should appear. Me/Profile must also have no Training videos row. If visible, check the installed commit/build and rebuild before changing production policy.
3. Follow the Aadhaar photo prompt with the consenting person's genuine document. Try cancel and one retake. A clear-photo result is only image quality, not admin approval. Failed/denied/cancelled capture must not count as uploaded.
4. Capture a clear genuine selfie. This becomes the private reference for later per-job face checks after human review. Retake/cancel should remain usable.
5. Enter the real receiving UPI or bank details through the existing screen. Invalid input should show an error. Valid syntax is not payout verification; the admin still reviews the details.
6. Continue to the existing rules/submission screen. Aman reads it and personally chooses whether to agree. Submission should become **under review**, not approved.
7. Complete the work profile: choose the actual served society/area and work periods. The time editor supports **4 AM-10 PM**, half-hour boundaries and breaks. Save, reopen and check the same periods restore. Work location/area and distance must fit the intended society; do not invent service coverage.

Training is intentionally hidden, not passed. No `trainingCompletedAt`, quiz pass or watch record should be fabricated to submit this application. Do not use a demo skip button, an approval timer or an operator data edit to bypass review.

Expected before approval: the application is visible to authorized admin, the Captain stays blocked from genuine Online/assignment, and corrections can be uploaded and resubmitted. The review timer never approves anyone.

6. Human admin approval and availability

Open https://wheelio-apps.web.app/admin/ in a browser and sign in with an approved verified Google reviewer. Captain phone OTP does not grant admin access.

Reviewer	Authorized Google account
Aman	amanmaheshwari900@gmail.com
Approved reviewer	ajay@trustkeyper.com
Approved reviewer	ajaybhatnagar1712@gmail.com
Approved reviewer	info@trustkeyper.com

1. Choose the real application under **Awaiting review**. Expect **Admin document review**, private identity photos, payout details and requested active societies.
2. The reviewer personally checks genuine identity, eligibility and payout details. Claude may explain the interface but must not make that human decision. Do not approve your own application; use another approved reviewer if required.
3. To test correction recovery, request a genuine needed correction with its item and note. Captain should see the note, edit that existing KYC stage and resubmit. A stale old browser decision should be rejected, not overwrite it.
4. Once genuinely acceptable, the reviewer selects the requested active societies, records the review note/confirmations and approves. If asked, refresh Google verification; recording a decision requires a recent reviewer sign-in.
5. Return to Captain. Approval/notification must arrive from Firebase. Tap Go online if still offline. If an earlier explicit request to go online was waiting, the app may retry it after approval. Confirm the server accepts it.

If Customer still sees no available slots

Check these in order: different Customer/Captain numbers; correct live-mode application; genuine admin approval; same approved society/area; saved work period overlap on the chosen date in India time; no active conflicting job or work restriction; current capacity; then retry/re-enter Customer time selection. Expired periods today cannot be booked. For an offer, keep Captain online in the foreground with a current connection/heartbeat; availability alone is not a job.

Do not fix availability by approving a fake Captain, expanding unknown coverage, editing booking timestamps, changing payment mode or bypassing work restrictions.

7. Customer: build a real booking

Have the approved Captain's society and working period ready first. For the same-session wash test, select a window that is open now or be prepared to wait for its start. Future reservations cannot be forced to Arrived/Started early.

1. On Customer Home choose the vehicle type, then the available wash option. The server supplies price/availability. Record the displayed total; no local sample amount or vehicle choice should override it.
2. Enter the real plate for the consenting vehicle. Test invalid/blank input. Optional Customer vehicle photos help locate it; they do not replace the Captain's four official before photos. Test camera cancel/retake and the designed optional-skip path if offered.
3. Select/check the actual service address or society. On the map, drag/confirm the vehicle's real parking pin; reopening should restore it. Typed unapproved addresses must show review/unserved behavior, not guessed coverage.
4. Enter tower/building, floor/basement and parking details accurately. Add an optional voice/video/text note if needed. Record, play and remove one note before using the final note; media uploads must finish before checkout.
5. Open **When should we come?** Choose date and available windows. Check the current all-day/multiple-window control when enough windows are open; it must select available windows, not manufacture capacity. Supported booking hours are 4 AM-10 PM; past/unavailable slots must remain unavailable.
6. Review vehicle, service, address, parking, time selection and total. Use Back to correct one item and check the rest survives. Then Continue to pay.

No-capacity and address-review states are valid blocks. Record the cause and resolve the actual approved Captain/work profile rather than changing designs or inventing a society. Different available windows can be selected by the server within the customer's requested choices; confirm the saved final window.

Expected: one persisted booking request with the correct Customer, amount, location, notes and chosen windows. Merely reaching checkout is not paid booking or confirmed Captain assignment.

8. Live payment and recovery

Real funds are involved. Before proceeding, Aman confirms the displayed amount, service and cancellation terms and personally authorizes any payment. Claude must not click through payment authorization or use downloaded API secrets. No merchant key, Firebase owner login or gateway-mode change is needed in the apps.

1. Open checkout. UPI/Google Pay/PhonePe, debit card and netbanking may be offered by the merchant. The Customer card choice should say **Debit card**. Direct credit-card checkout/EMI/pay-later must not be offered as an accepted option.
2. If an unsupported direct credit-card option appears, stop that payment and record FAIL. Do not charge a credit card to see whether recovery works. A credit card linked within UPI is allowed under the owner's chosen policy.
3. First cancel checkout before paying. Return to the booking and retry safely. The app must not mark the order paid or dispatch a Captain from cancellation.
4. For the approved live attempt, Aman chooses the allowed method, verifies the amount and completes bank/UPI authorization himself. Test native app switch and return. An SDK success screen alone must not be treated as server capture.
5. Customer should move to the paid/searching/assignment state after verified payment. If the bank paid but the callback is lost, reopen the existing booking and use its payment-status recovery. **Do not immediately pay again or create another booking.** Retain the booking/payment reference privately for review.
6. Reopen Customer and Captain. The same paid order should remain; retries must not create duplicate charges, capacity reservations or dispatches.

Use the current live WashKrew merchant. Historical test transactions/refunds keep their original merchant and mode. Never relabel an old order or switch the global production catalog to test just for phone acceptance. Simulated payment checks belong in isolated local QA, with an explicit simulated result label.

Expected: one captured payment mapped to one actual booking. A failed, cancelled or unresolved payment must not be shown as successful. A captured unsupported method is a recovery/refund case, not a dispatchable paid job.

9. Captain: accept, arrive and take before photos

Keep Captain online and foreground during assignment. Its saved periods, area, society and mode must fit the actual booking. Customer should show the same assigned Captain/order once the offer is accepted.

1. When the real offer rings/appears, inspect its allowed preview, service and pay. Accept that offer once. Expired/rejected/already-claimed offers must not become an accepted job. Exact private parking/notes belong to the assigned Captain, not unrelated accounts or an unaccepted offer.
2. Confirm Customer changes from searching to assigned/on the way and both apps show the same vehicle/society/window. Reopen the Captain app and recover the active job instead of starting a sample lifecycle.
3. Optionally enable foreground location. Move safely at the actual service location and check Customer receives recent position/ETA when available. Route-provider errors or stale data must not become invented arrival times.
4. At the actual vehicle, after the selected window opens, tap the arrival action. The server must reject premature arrival/start. If there is a gate/vehicle problem, use the existing help path instead of faking arrival.
5. If an owner voice/video note exists, listen/watch through the required note step. Verify it is the actual uploaded note, not a sample sound.
6. Use **Quick check -> Open camera** for the consenting approved Captain's fresh selfie. Cancel and retry once. Poor/uncertain results must offer retry or operations review, not automatic KYC approval. Start a fresh attempt if its five-minute capture session expires.
7. Capture the four official before sides in their correct boxes. Try one retake and reopen the screen. Cancelled, denied, dark/unreadable or failed uploads must not fill a successful evidence slot. All four are required to start.

Expected: only the assigned approved Captain can operate the job. The fresh identity check is separate from admin application approval. A rejected/uncertain selfie must not be worked around with another person's face or a database edit.

10. Wash, tracking, photos and Customer approval

1. Captain starts the wash after arrival, identity and four before photos. Customer should see Washing for the same order. Timing/status must follow persisted server transitions rather than sample timers.
2. While Captain is foreground, inspect Customer location/ETA on the companion browser/phone. Background or lock Captain briefly, then reopen. Stale location must be clear; this app does not promise continuous background GPS. With one phone, switching to Customer also backgrounds Captain, so record that limit.
3. Finish the real consenting vehicle wash. Capture all four after sides. Retake the correct side and check it survives reopening. Incomplete uploads must block finish. A network/upload failure must display an error and allow retry.
4. Tap **Check photos**. Real advisory review can flag a side or be unavailable. Review/retake the indicated side, or use the existing acknowledged send-anyway path if appropriate. Neither a clean AI result nor a send action approves the wash.
5. Send the complete evidence. Customer should receive photos-ready/review state; Captain earnings should be awaiting approval. Reopen both apps to verify it persists, then compare before/after views and the actual private photos.
6. The Customer personally inspects the result and taps **Approve**, or follows Report a problem for a real concern. Do not fabricate a dispute or force a customer decision. Another account/Captain cannot approve this Customer's wash.
7. Rate/tag the completed wash. Repeated approval/rating must not award duplicate earnings or completion counts. Verify the booking is in history and a receipt/ evidence download works only for its authorized party.
8. Open Captain Money/jobs history. Confirm the correct order and approved earning. An approved ledger entry is **not proof of a bank transfer**. Actual bank payouts are human operations and must not be marked sent by a test timer.

Check notification/inbox links open the correct booking/job. Test both native foreground and background push only after APNs is configured (page 12). Before calling this journey PASS, record the same order reference and actual observed statuses from both roles, with private evidence kept out of shared reports.

11. Cancellation, refund and problem handling

Use a separate, expressly approved real booking for a refund check. Confirm the displayed amount and cancellation terms first. Aman authorizes its payment and the actual cancellation himself. Prefer a refundable pre-assignment cancellation to avoid an unexpected assigned-job fee; do not assume every cancellation is free.

Refund check

1. Pay the agreed additional booking, then cancel it through the Customer app under the applicable displayed terms. Confirm no active Captain wash controls remain for the cancelled order and it cannot later be claimed/started.
2. Open **See refund / Track refund**. Record the amount, original payment method and actual state. Reopen or check status again; retries must not issue duplicate refunds. Do not edit the provider ID or create a second manual API refund.
3. Interpret states accurately: Queued means waiting; Sent to Razorpay means accepted by provider; Processed means Razorpay confirmed it. Bank posting may still take time. Delayed/Needs review means the money remains due, not refunded.
4. If the provider stalls/rejects, record BLOCKED/FAIL as appropriate and ask the responsible operations person to inspect that original payment/refund. Preserve its original merchant/mode; do not mark it processed by hand.

Configured historical wallet/free-wash/credit entitlements must retain their existing rules. Do not seed balances or manufacture a free wash for live testing. Original-method cash refunds must not be redirected to a newly typed UPI ID.

Reports and disputes

For a genuine concern, Customer uses Report a problem; verify the private report, Captain response where available, and held/disputed earnings. A human resolves the outcome and any actual refund. The Captain admin page is for applications; do not invent an unimplemented dispute-review page. Owner tooling is documented in `backend/firebase/operator.cjs`; Claude may prepare a redacted case for the responsible reviewer, never decide eligibility, a dispute or a bank payout itself.

If no genuine disputed case is available, exercise the branch in isolated QA and record it as simulated. Do not manufacture a complaint against a real worker.

12. iPhone permissions, settings and recovery

Use iOS Settings for the actual installed app when changing permissions. Test one denial/cancel/re-enable cycle at a time; return to the same screen to retry.

Area	Required observation on each relevant app
Camera/photos	Allow, cancel, deny and re-enable; KYC/selfie, vehicle and all before/after captures remain usable. Failed capture is never a completed upload.
Microphone/media	Customer voice/video note records, previews, removes and retries; assigned Captain plays actual private media. Denial explains how to recover.
Location/maps	Real map/pin restores; denied permission does not invent coordinates. Foreground sharing/ETA works; background/stale behavior is honest.
Notifications/APNs	Allow/deny/re-enable. Test native foreground/background delivery, correct tap destination, settings and persisted private inbox. Token/sign-out behavior must not leak jobs.
App/network lifecycle	Reopen each cold Release app and its active booking; briefly lose connection during a safe step, then retry the same record without duplicate booking/payment/upload.
Account/settings	Profile photo, language, work periods/area and payout display restore. Payout or phone changes may require fresh SMS verification.

Training and quiz are hidden, so speech/quiz playback is not part of current acceptance. Do not restore the hidden screens to obtain a checklist pass.

Test log-out/log-in only after recording active order references. Check denied push still leaves the private inbox usable. Bank payout change/verification is not proven by a syntactically valid UPI ID or saved field.

Optional last step: account deletion. Use a disposable consenting account with no open jobs/refund/dispute obligations, only if Aman explicitly wants to delete it. Aman confirms deletion himself. Current-app-only deletion must preserve any other role's account and obligations. This destructive step is not needed to prove the primary booking; never delete an active production account as cleanup.

Missing Apple provisioning/APNs access, service coverage, support contacts or approved legal policies are external/setup blocks. Record them without inventing phone numbers, agreements, downloads, background location or fake push success.

13. Claude: diagnose failures without redesigning

Symptom	Check before changing code
Training still visible / old layout	Correct staging commit, rebuilt Customer/Captain workspace and installed app; training flags are disabled. Do not restore old demo UI or change the design.
Xcode/signing/pod failure	Exact workspace, selected team/bundle ID, phone trust/iOS support and the named missing/incompatible pod. Preserve locks and record the first real error.
OTP fails	Actual different mobile number, SMS quota/cooldown, device connectivity, Firebase native app/URL scheme and any reCAPTCHA flow. No fixed OTP or invented success.
Admin page denied	An exact approved verified Google account, correct browser profile and recent Google verification. Phone OTP is not reviewer access.
Cannot submit KYC	Actual completed uploads, valid payout details, requested active society, latest server application state and correction items. No fake completion flags.
Approved but no slots/offers	Same live mode/society/area, real approval, distinct Customer ID, saved work-period overlap, capacity/conflicts, work restrictions and online/heartbeat for offers.
Paid but order not confirmed	Existing booking reference and server payment-status recovery. Preserve provider/order IDs; do not pay again, relabel or regenerate keys.
Wash cannot start/finish	Selected window has opened, assigned Captain, arrival, current selfie session, all four correct before/after uploads and photo-review acknowledgement.
Tracking/push fails	Foreground/fresh location vs background; actual Routes/network status; native Apple entitlement/provisioning/APNs vs browser FCM.
Money/refund looks wrong	Server order/earnings/refund state and original merchant/mode. A human checks provider rejection, dispute decision or actual bank transfer.

Reproduce in the existing screen, change the smallest cause, run relevant checks, then have Aman repeat the physical action. Every app fix must land in Flutter. Update the acceptance record and commit working Mac locks/project changes.

Current non-device inputs still include approved support/legal details and live commercial/coverage confirmation. DigiLocker, WhatsApp Business, masked calling and voice-call OTP remain deferred; do not call them connected. Training content is not a current blocker under the owner's explicit hidden-training decision.

14. Results Claude must leave for Aman

Create `docs/IOS-ACCEPTANCE-YYYY-MM-DD.md` on the Mac. Use PASS, FAIL or BLOCKED for every row. A source test, web preview, simulated capture or successful compile does not count as an observed physical-device pass.

Case	Result and evidence to record
I01 Build/install	Both signed Release apps; exact commit, Xcode/Flutter/iOS versions and phone model
I02 Customer account	Real SMS, profile, incorrect-code handling and cold reopen
I03 Captain account	Different mobile number, real SMS, offline/unapproved state
I04 KYC/policy	Three KYC stages, no training/quiz; genuine uploads and payout validation
I05 Admin approval	Real human review, correction if needed, approval before Online
I06 Availability/booking	Correct society/period, all-day/multiple windows, saved parking/media
I07 Live payment	Aman-authorized amount/method, UPI switch/return, no direct credit-card checkout
I08 Assignment/wash	One accepted Captain; arrival, fresh selfie, four before/four after uploads
I09 Tracking/review	Actual foreground/stale behavior, photo review, Customer approve/rate and earnings
I10 Cancellation/refund	Terms/amount, no stale wash controls, actual provider state, no duplicate refund
I11 Permissions/push	Camera/media/location recovery, APNs foreground/background and correct taps
I12 Persistence/settings	Reopen, private inbox/settings/work periods, sign-out and optional deletion scope

For each failure include: role/screen, numbered steps, expected vs actual result, time/timezone, redacted error/evidence, owning blocker, fix commit and retest result. Keep real phone numbers, OTPs, card details, KYC images and precise private parking out of the shared document. Keep necessary transaction references private.

Final handoff back to the owner

Report what passed on hardware, what failed or is blocked, exact commits changed, actual live booking/refund outcome and any Apple/content/operations input still needed. Do not say production-ready while a required case is unverified. End with the next concrete action for each open item; preserve the staging design.